

PHASE 7

Project Documentation Phase

EduGenie - AI Powered Learning Assistant

Assemble the technical and user-facing documentation required for maintenance, evaluation, and future enhancement.

Prepared as a phase-wise academic submission document based on the supplied EduGenie-Ai project archive.

1. Project Overview

EduGenie is designed as a browser-based educational assistant using FastAPI for server-side routing and Google Gemini for AI-generated learning content. The planned feature set includes question answering, explanations, quizzes, summarization, and learning recommendations.

2. Project File Documentation

main.py	FastAPI application entry point, routes, templates, static-file mounting, and exception handling.
requirements.txt	Python package constraints for runtime and testing dependencies.
.env	Environment-based AI configuration such as model name and temperature; contains a credential in the supplied archive and should be treated as sensitive.
templates/index.html	Expected web page template; currently empty in the supplied snapshot.
static/css/style.css	Responsive user-interface styles.
static/css/js/app.js	Task selection, request dispatch, result rendering, quiz interaction, status messages, and copy behavior.
Missing modules referenced by main.py	Expected task logic and data-model implementations.

3. User Guide

Open the EduGenie web interface. Select a task such as Question Answering, Explanation, Quiz, Summarize, or Learning Recommendations. Enter the question, topic, or passage. For Learning Recommendations, select the learner level. Press Generate and review the output. Use Clear to reset the form and Copy to copy the visible result.

4. Developer Setup Guide

Typical local setup flow:

```
python -m venv .venv .venv\Scripts\activate pip install -r requirements.txt uvicorn main:app --reload
```

Before starting the application, configure the Gemini API credential through the environment. Do not place real credentials inside the final academic report or publicly shared archive.

5. API Documentation Summary

/qa	text	TextResponse.result
/explain	text	TextResponse.result
/quiz	text	QuizResponse.result.questions
/summarize	text	TextResponse.result
/learn/recommendations	text + level	LearningPathResponse

6. Maintenance Notes

- Keep the FastAPI route names synchronized with the frontend endpoint map in app.js.
- Keep response field names synchronized with result rendering functions.
- Update dependencies using tested version ranges and re-run the complete test suite after changes.

- Record model changes, prompt changes, and API configuration changes in the project documentation.
- Remove development secrets before submitting or publishing the project.

7. Known Gaps in Supplied Archive

The current archive is best interpreted as an implementation snapshot rather than a complete final release. The documentation therefore distinguishes the observed files from the intended module architecture. A final submission should include the missing modules and a populated index.html, along with evidence that the complete system runs.

8. Documentation Checklist

- Problem statement and objectives
- Requirements and architecture
- Database/API/data-model description as applicable
- Implementation notes
- Test cases and results
- User manual and setup instructions
- Known limitations and future enhancement plan